20.LLAMA

24 March 2026 15:04

Collection of LLMs → 7B to 65B

train SOTA models using publicly available datasets
(no proprietary & inaccessible datasets)

smaller models + more data
↳ competitive or better performance
Kaplan → scale params
Chinchilla → Scale params + data jointly

Training compute ≠ inference cost
Optimize for inference cost under a target performance

Kaplan et al (Scaling models to a sufficient size)
↓
Chinchilla Hoffman et al 2022
(given compute budget,
best performances are achieved by smaller models trained on more data)

↳ determine how to best scale the dataset and model sizes for a particular training compute budget

↓ it disregards the inference budget, which is critical when serving an LM at scale

↓ In this context, given a target level of performance → preferred model isn't the fastest to train but the fastest at inference

Chinchilla
N, D → not "equally" in magnitude
But optimally according to a compute trade off curve
tokens & parameters
(not necessarily equal, but balanced)

It may be cheaper to train the larger model to reach a certain level of performance,
a smaller one trained longer will ultimately be cheaper at inference

Focus → train a series of models that achieve the best possible performance at various inference budgets, by training on more tokens than typical

LLaMA → 7B to 65B
LLaMA 13B >> GPT-3 being 10X smaller

They mention performance continues to improve after 1T tokens
↳ scaling behavior signal
intentionally overtraining relative to chinchilla optimum

Architecture (compared to Vaswani)

① Pre Normalization
(to improve training stability, we normalize input of each transformer sub layer, instead of the output)

RMSNorm Normalizing function (Zhang 2019)

Objective
Maximize performance
Subject to inference constraints } not training optimal but deployment optimal

Pre Norm → normalization before sublayers
Mechanism view
Pre LN changes gradient flow stability

② SwiGLU activation function

Replace ReLU with SwiGLU Shazeer (2020)
Use a dimension of $\frac{2}{3}$ 4d instead of 4d in PaLM

Gradient Lens →
Post LN → gradients must pass through residual + norm → unstable at depth
Pre LN → Identity path exists → clean gradient flow
(without pre LN → deep models become harder to train)

③ Rotary Embeddings [GPTNeo]

Remove absolute positional embeddings
add rotary positional embeddings
(RoPE) introduced by Su et al (2021)
at each layer of the network

→ Layer Norm → normalizes mean + variance
→ RMSNorm → normalizes only magnitude (no mean centering)
↳ Removes unnecessary constraint → less computation → more stable scaling

ReLU → single gate
GLU → multiplicative gating
SwiGLU → smoother + better gradient flow
↳ Adds learned gating → selective feature propagation

Here they use $\frac{2}{3} \times 4d$ instead of

AdamW optimizer with $\beta_1 = 0.9$, $\beta_2 = 0.95$
Cosine - learning rate schedule
such that final learning rate = 10% of maximal learning rate

Implementation –

(1) efficient implementation of the causal multi head attention to reduce memory usage & runtime

No storing of attention weights

to reduce memory usage & run time

No storing of attention weights
No computation of key/query scores that are masked
due to causal nature of language modeling task

→ reduce the no of activations recomputed during the backward pass with checkpointing
↳ Save the activations that are expensive to compute, such as the outputs of linear layers
↳ Achieved by manually implementing the backward function for the transformer instead of relying on pytorch autograd

→ To fully benefit → reduce the memory usage of the model by using
Model & Sequence parallelism (Korthikanti et al 2022)

→ Overlap the computation of activations and communication between GPUs over network as much as possible

Here they use $\frac{8}{3} \times 4d$ instead of $d_{ff} = 4d$ in standard transformer

design tradeoff { SwiGLU increases parameter usage
So they reduce width to keep compute constant

→ They dont store attention wts, dont compute masked parts

They avoid → storing full $n \times n$ attention matrices
→ computing masked future tokens

Memory $O(n^2)$ (they address this)

Checkpointing (save expensive activations)

Tradeoff Memory ↓ compute ↑

memory v compute trade off

Why Custom Backward?
Auto grad = general but inefficient
Custom backward = control memory + recomputation

Reason GELU
↳ smooth activation unlike ReLU
↳ keeps small negative values partially alive

ReLU → hard cut off
GELU → soft gating (probabilistic like filtering)

BERT/GPT → GELU (smooth scalar activation)
LLaMA → SwiGLU (gated activation)
GELU → element wise non linearity
SwiGLU → learned gating b/w features

Moving from passive non linearity → active feature selection

LLaMAs efficiency comes from

memory aware attention
activation checkpointing
distribution parallelism
enabling large scale training

* mean centering → less critical in residual architectures

RMS Norm → only magnitude
Layernorm
→ $RMS(x) = \sqrt{\frac{1}{d} \sum x_i^2}$

$y = \frac{x}{RMS(x)} \gamma$

Cheaper computation, simpler, works at scale

ReLU → $RELU(x) = \max(0, x)$
Kills negative values completely
loses information

GELU → $x\,\phi(x)$ probabilistic gating

GLU → instead of activation of x
we do $(xW_1)\,\sigma(xW_2)$

One part = content
One part = gate

$SwiGLU(x) = (xW_1)\,Swish(xW_2)$
$Swish(x) = x\,\sigma(\beta x)$

Information flow lens
↓
model learns what to pass instead of blindly transforming everything

GLU doubles parameters
so they shrink width to compensate

Model parallelism → split model across GPUs (params)
Sequence parallelism → split sequence across GPU's (activations across tokens)
} Compute + Communication together
Reduce idle gpu time

RMSNorm points
Mean subtraction is not essential for transformers
Removing it
↓ compute
↓ numerical noise
↑ stability at scale

RoPE → mechanistic difference

instead of $x + p$ rotates query & key vectors based on position

Attention uses QK^T, without position → no order awareness

Absolute embeddings $x + P$ (added externally)

RoPE → instead of modifying input
it modifies interaction

$$Q \rightarrow R_\theta(Q)K \rightarrow R_\theta(K)$$

↓
rotation depending on position

Rotation encodes relative position as phase difference

Attention becomes $Q_i K_j$ → depends on $i-j$

Absolute → token is at position 10
RoPE → token is 5 steps away from j

Scaling laws → works for longer seq

Information flow → position is inside attention, not added outside